TUIA NLP 2026

Trabajo Práctico Final

Asistente Virtual de Electrodomésticos — ElectroHogar

Alumno: Maximiliano Francisco Chiappe
Legajo: C-7542/6
Materia: Procesamiento del Lenguaje Natural
Fecha de entrega: 23/06/2026

Repositorio: https://github.com/MaxiChiappe/TP_NLP_CHIAPPE_Maximiliano.git

1. Introducción

El presente trabajo práctico desarrolla un asistente virtual especializado en información de una empresa de electrodomésticos denominada ElectroHogar. El sistema es capaz de responder preguntas sobre uso de productos, precios, opiniones de usuarios, recomendaciones y relaciones entre categorías, integrando múltiples fuentes de datos heterogéneas mediante el paradigma RAG (Retrieval Augmented Generation).

El proyecto implementa dos ejercicios principales. El primero consiste en un sistema RAG con tres bases de datos diferenciadas: vectorial, tabular y de grafos. El segundo transforma ese sistema en un Agente ReAct capaz de razonar sobre qué herramientas usar para responder consultas complejas. Todas las tecnologías utilizadas son de código abierto y se ejecutan completamente local dentro de Google Colab, sin depender de APIs externas de ningún tipo.

El paradigma RAG, introducido por Lewis et al. (2020), propone combinar un componente de recuperación de información con un modelo generativo de lenguaje. En vez de depender únicamente del conocimiento almacenado en los parámetros del modelo durante el entrenamiento, RAG recupera información relevante de bases de datos externas y la incorpora al contexto del LLM en tiempo de inferencia. Esto permite responder preguntas sobre información específica del dominio sin necesidad de fine-tuning del modelo.

El dataset provisto por la cátedra contiene información variada sobre productos, ventas, soporte técnico, manuales y reseñas de usuarios de ElectroHogar, lo que permite demostrar las capacidades del sistema ante distintos tipos de consultas que requieren diferentes estrategias de recuperación.

2. Fuentes de Datos

El dataset utilizado provee información estructurada y no estructurada de ElectroHogar. Se compone de archivos en múltiples formatos que representan distintos aspectos del negocio de electrodomésticos:

- productos.csv: 300 productos con descripciones técnicas, precios en USD, stock disponible y características como potencia, capacidad y voltaje
- vendedores.csv: registros de vendedores con nombre, apellido, sucursal asignada y nivel de experiencia
- ventas_historicas.csv: 10.000 transacciones con fecha, producto, vendedor, sucursal, cantidad, precio unitario, descuento y método de pago
- inventario_sucursales.csv: 4.100 registros de stock por sucursal con precios locales y fechas de reposición
- tickets_soporte.csv: 2.000 tickets de soporte técnico con problema reportado y estado de resolución
- devoluciones.csv: 800 registros de devoluciones con motivo, estado y monto de reembolso
- faqs.json: 3.000 preguntas frecuentes organizadas por producto con categoría y respuesta
- manuales_productos/: 50 manuales técnicos en formato Markdown con especificaciones detalladas
- resenas_usuarios/: 5.015 reseñas individuales en formato texto con puntaje y comentario del usuario

2.1 Diseño de las Tres Bases de Datos

A partir del dataset se diseñaron tres repositorios de información con estructuras y propósitos diferenciados. La decisión de qué información va en cada base se tomó considerando el tipo de consulta que cada base debe servir eficientemente:

Tipo de Base	Datos Almacenados	Justificación
Base Vectorial (ChromaDB)	Descripciones, FAQs, manuales, reseñas	Búsqueda por significado semántico
Base Tabular (SQLite)	Productos, precios, ventas, inventario, devoluciones	Filtros exactos y consultas numéricas
Base de Grafos (Neo4j)	Relaciones categoría-producto-marca-sucursal-vendedor	Navegación de relaciones entre entidades

2.2 Justificación del Diseño

La base vectorial es adecuada para las fuentes de texto no estructurado donde el significado semántico importa más que la coincidencia exacta de palabras. Un usuario que pregunta '¿cómo hago smoothies?' necesita encontrar documentos sobre 'uso de licuadora' aunque no use esas palabras exactas. La búsqueda por embeddings permite capturar esta similitud semántica de forma natural.

La base tabular es adecuada para información estructurada con valores numéricos. Preguntas como '¿cuáles son las licuadoras de menos de $200?' requieren filtros exactos por precio que SQL maneja eficientemente con condiciones WHERE, rangos y agregaciones COUNT/SUM. Intentar responder esto con búsqueda semántica sería impreciso e ineficiente.

La base de grafos es adecuada para modelar relaciones entre entidades. Preguntas como '¿qué productos están relacionados con la categoría Cocina?' o '¿qué vendedores trabajan en Mendoza?' son inherentemente relacionales. Un grafo las representa de forma natural permitiendo navegación eficiente de relaciones sin necesitar JOINs complejos como en SQL.

Esta separación en tres bases de datos heterogéneas requiere un clasificador de intención que determine cuál base consultar para cada pregunta del usuario, lo cual se implementa en el item 1.5.

3. Ejercicio 1: Sistema RAG

El sistema RAG implementado sigue el diagrama de flujo propuesto en la consigna: la consulta del usuario pasa por el clasificador de intención, que determina si se consulta la base vectorial (con búsqueda híbrida + ReRank), la base tabular (con generación dinámica de SQL) o la base de grafos (con generación dinámica de Cypher). El contexto recuperado se incorpora al prompt del LLM para generar la respuesta final.

3.1 Base de Datos Vectorial (ChromaDB)

Se implementó ChromaDB como base de datos vectorial con almacenamiento persistente local en /content/chroma_db. Los documentos se dividen en fragmentos mediante RecursiveCharacterTextSplitter y se codifican con un modelo de embeddings multilingüe antes de ser indexados en la colección.

3.1.1 Modelo de Embedding

Se utilizó el modelo paraphrase-multilingual-MiniLM-L12-v2 de la biblioteca sentence-transformers. Este modelo pertenece a la familia MiniLM, que aplica destilación de conocimiento para comprimir modelos más grandes manteniendo buena calidad. La elección se justifica por los siguientes motivos:

- Soporte nativo para español: el modelo fue entrenado con datos en más de 50 idiomas incluyendo español, lo que garantiza buena calidad semántica para el dominio del TP
- Eficiencia computacional: con solo 12 capas transformer y 118M de parámetros, puede ejecutarse eficientemente en la GPU T4 de Colab gratuito sin agotar la memoria
- Calidad semántica demostrada: el modelo fue específicamente entrenado para paráfrasis multilingüe, lo que significa que frases con el mismo significado en distintas palabras tendrán vectores similares
- Dimensionalidad de 384: balance adecuado entre expresividad vectorial y eficiencia de almacenamiento. Modelos más grandes generan vectores de 768 o 1024 dimensiones con mayor consumo de memoria
- Disponibilidad: se descarga automáticamente de Hugging Face sin necesitar credenciales ni API keys

3.1.2 Text Splitter

Se configuró RecursiveCharacterTextSplitter de LangChain con los siguientes parámetros justificados:

- chunk_size=500: cada fragmento contiene hasta 500 caracteres. Este tamaño es suficiente para preservar el contexto completo de una pregunta-respuesta de FAQ o un párrafo de manual técnico, sin exceder la ventana de contexto del modelo de embedding
- chunk_overlap=50: superposición de 50 caracteres entre fragmentos consecutivos para evitar pérdida de información cuando una idea importante cae justo en el límite entre dos chunks. Sin este solapamiento, una oración dividida entre dos chunks perdería contexto en ambos extremos

El uso de RecursiveCharacterTextSplitter en particular (en vez de CharacterTextSplitter simple) es importante porque intenta dividir respetando la estructura natural del texto: primero intenta dividir en párrafos, luego en oraciones, y solo como último recurso divide en el límite exacto de caracteres. Esto produce chunks más coherentes semánticamente.

Esta configuración resulta en 9.456 chunks distribuidos entre las cuatro fuentes indexadas:

Fuente	Chunks	Tipo de contenido
Productos	613	Descripciones técnicas de productos
FAQs	3.000	Preguntas y respuestas frecuentes
Manuales	828	Documentación técnica detallada
Reseñas	5.015	Opiniones de usuarios
Total	9.456	

3.1.3 Configuración de ChromaDB y Similitud Coseno

Se configuró ChromaDB con la métrica de similitud coseno (hnsw:space: cosine) en lugar de la distancia euclidiana que usa por defecto. La similitud coseno mide el ángulo entre dos vectores en el espacio multidimensional, independientemente de su magnitud. Esto es más apropiado para embeddings de texto porque dos documentos sobre el mismo tema pero de distinta longitud tendrán vectores similares en dirección aunque no en magnitud.

Cada fuente se indexa con metadata de tipo (fuente: faq, manual, resena, prod) y campos adicionales como id de producto, nombre y categoría. Esta metadata permite filtrado posterior por tipo de documento, lo cual se usa en el chatbot cuando el usuario pregunta por opiniones (filtro fuente='resena') o instrucciones (sin filtro para buscar en manuales y FAQs).

3.2 Base de Datos Tabular (SQLite)

Se implementó SQLite en memoria RAM utilizando pandas para cargar los datos desde CSVs y la librería sqlite3 de Python para ejecutar consultas SQL dinámicas. Se crearon 5 tablas: productos, inventario, ventas, devoluciones y vendedores. La elección de SQLite en memoria se justifica porque es más rápida que en disco para consultas repetidas, y como los datos se recargan automáticamente desde CSV en cada sesión, no se necesita persistencia.

3.2.1 Schema Info para el LLM

Para que el LLM pueda generar SQL correcto sin ver los datos completos (lo cual sería inviable por la cantidad de tokens), se construye un string de información del esquema que incluye: nombres de todas las columnas de cada tabla, valores únicos de las primeras 4 columnas de texto (para que el LLM conozca los valores válidos de categorías, métodos de pago, etc.) y rango mínimo-máximo de las primeras 4 columnas numéricas (para que el LLM sepa los rangos válidos de precios y stock).

3.2.2 Generación Dinámica de SQL con Few-Shot

El LLM local genera consultas SQL a partir de lenguaje natural usando few-shot prompting con 7 ejemplos concretos de pregunta-SQL. Los ejemplos muestran el estilo de consultas esperado: uso de LOWER(nombre) LIKE para búsqueda por nombre de producto, ORDER BY para ordenamiento, LIMIT para limitar resultados y COUNT(*) para conteos.

Se implementó un mecanismo de fallback basado en palabras clave que construye el SQL manualmente cuando el LLM genera consultas con errores sintácticos. Este fallback detecta

9 productos clave (licuadora, cafetera, heladera, etc.) y modifica la consulta según si el usuario pregunta por el más barato, el más caro o simplemente disponibles.

3.3 Base de Datos de Grafos (Neo4j)

Se implementó Neo4j Community Edition ejecutado localmente dentro del entorno de Colab usando Java 11, cumpliendo con el requisito de ejecución sin APIs externas. La conexión se realiza mediante el protocolo Bolt en el puerto 7687 (puerto estándar de Neo4j), con autenticación desactivada para simplificar el acceso en el entorno local.

El grafo modela las siguientes relaciones entre entidades del dominio de ElectroHogar, importadas desde los DataFrames usando MERGE para evitar duplicados:

- (:Producto {id, nombre, precio})-[:PERTENECE_A]->(:Categoria {nombre})
- (:Producto {id, nombre, precio})-[:PERTENECE_A]->(:Subcategoria {nombre})
- (:Producto {id, nombre, precio})-[:ES_DE_MARCA]->(:Marca {nombre})
- (:Vendedor {id, nombre})-[:TRABAJA_EN]->(:Sucursal {nombre})
- (:Producto {id, nombre, precio})-[:DISPONIBLE_EN]->(:Sucursal {nombre})

3.3.1 Consideración Técnica de Instalación

Se identificó un problema crítico durante el desarrollo: si el servidor Neo4j (carpeta /content/neo4j) se descarga antes de instalar el driver Python (paquete neo4j), Python confunde la carpeta del servidor con el paquete y falla el import con 'cannot import name GraphDatabase from neo4j (unknown location)'. La solución fue instalar el driver Python primero y luego descargar el servidor.

3.3.2 Generación Dinámica de Cypher

Las consultas Cypher se generan usando dos enfoques: la función buscar_grafo_llm() usa el LLM con few-shot prompting para máxima flexibilidad, y la función buscar_grafo() usa detección de palabras clave para máxima robustez. En el pipeline principal se usa buscar_grafo() porque el modelo de 3B parámetros comete errores tipográficos frecuentes en los nombres de relaciones Cypher.

Se implementó un diccionario de corrección de nombres de relaciones mal escritos: ES_DE_MARBKA → ES_DE_MARCA, TRABAJA_IN → TRABAJA_EN, TRABAJO_CON → TRABAJA_EN. Estos errores específicos se detectaron durante las pruebas y se documentan como limitación del modelo pequeño.

3.4 Clasificador de Intención

El clasificador de intención es el componente que enruta cada consulta del usuario a la base de datos más apropiada. Sin este clasificador el sistema tendría que consultar las tres bases siempre, lo cual sería lento e ineficiente. Se implementaron y compararon dos enfoques según lo solicitado en el item 1.5.

3.4.1 Clasificador Entrenado (TF-IDF + Regresión Logística)

Se entrenó un clasificador de texto usando scikit-learn con un Pipeline que encadena TF-IDF (vectorización) y Regresión Logística (clasificación). El conjunto de entrenamiento consiste en 49 preguntas sintéticas generadas manualmente: 15 de clase vectorial (sobre uso y reseñas), 19 de clase tabular (sobre precios y stock) y 15 de clase grafos (sobre categorías y marcas).

TF-IDF (Term Frequency-Inverse Document Frequency) convierte cada pregunta en un vector numérico ponderando cada palabra según qué tan frecuente es en esa pregunta y qué tan rara es en el corpus completo. Esto permite identificar palabras clave discriminativas como 'precio' y 'cuesta' para tabular, 'opinan' y 'reseñas' para vectorial, y 'categoría' y 'marca' para grafos.

3.4.2 Clasificador Few-Shot (LLM)

Se implementó un clasificador basado en el LLM local utilizando few-shot prompting. El modelo recibe en el prompt la descripción de las 3 categorías y 8 ejemplos de preguntas ya clasificadas, y debe responder con una sola palabra: vectorial, tabular o grafos. Se usa max_tokens=10 porque solo se necesita una palabra como respuesta.

La principal ventaja de este enfoque es que no requiere datos etiquetados para entrenamiento y puede adaptarse a nuevas categorías simplemente agregando ejemplos al prompt. La desventaja es su menor precisión con modelos pequeños y su mayor latencia por requerir una llamada al LLM.

3.4.3 Comparación de Clasificadores

Se evaluaron ambos clasificadores con 15 preguntas de prueba no vistas durante el entrenamiento (5 por clase), obteniendo los siguientes resultados:

Modelo	Accuracy	Precision	Recall	F1-Score
Clasificador Entrenado (TF-IDF + LR)	0.93	0.94	0.93	0.93
Clasificador Few-Shot (Qwen 2.5 3B)	1.00	1.00	1.00	1.00

En esta ejecución el clasificador Few-Shot obtuvo 100% de accuracy (1.00 en todas las métricas), superando al clasificador entrenado (93%). Sin embargo, el clasificador Few-Shot es no determinístico — depende del LLM con temperature > 0 y puede variar entre ejecuciones. El clasificador entrenado es más confiable para producción por ser completamente determinístico y reproducible. Se elige el clasificador entrenado como principal en el pipeline por esta razón.

El clasificador Few-Shot sería preferible en dominios donde no se dispone de datos etiquetados o donde el vocabulario es muy variable. Para este TP el clasificador entrenado es claramente superior dado el dominio acotado y el vocabulario predecible de las consultas.

3.5 Pipeline de Recuperación: Búsqueda Híbrida + ReRank

Se implementó un pipeline de búsqueda híbrida que combina dos enfoques complementarios para maximizar la calidad de los resultados recuperados. Ninguno de los dos enfoques por sí solo es suficiente: la búsqueda semántica puede traer documentos relevantes por significado pero pasar por alto términos exactos del dominio, mientras que BM25 puede perder documentos semánticamente relacionados.

- Búsqueda semántica (ChromaDB): encuentra documentos por similitud coseno de embeddings. Busca el significado aunque se usen palabras distintas a la consulta
- BM25 (Best Match 25): algoritmo probabilístico clásico basado en frecuencia de términos con normalización por longitud del documento. Encuentra documentos que contienen exactamente las palabras de la consulta

El proceso de ReRank combina ambos scores mediante normalización y media ponderada 50%/50%, reordenando los resultados por score final. Para cada búsqueda se recuperan k*2 candidatos de cada fuente y se devuelven los k mejores del pool combinado. Un fragmento que aparece en ambas búsquedas acumula ambos scores, siendo naturalmente favorecido en el ranking final.

3.6 Checkpoint de Recuperación (Item 1.7)

Se implementó el checkpoint obligatorio que verifica el pipeline completo de recuperación para los 5 prompts especificados en la consigna, antes de integrar la generación con el LLM. El checkpoint muestra para cada prompt: la intención detectada, la fuente seleccionada, la consulta generada (SQL o Cypher si corresponde) y los fragmentos recuperados con su contenido, fuente, metadata y score.

Prompt	Intención	Método	Resultado
¿Cómo uso mi licuadora para hacer smoothies?	VECTORIAL	Híbrida + ReRank	FAQs de licuadora (score: 0.50)
¿Cuáles son las licuadoras de menos de $200?	TABULAR	SQL dinámico	DataFrame vacío (no hay < $200)
¿Qué opinan los usuarios de esta cafetera?	VECTORIAL	Híbrida + ReRank	Descripciones de productos
Quiero una licuadora con buenas reseñas	VECTORIAL	Híbrida + ReRank	Reseñas de licuadoras
¿Qué productos están relacionados con Cocina?	GRAFOS	Cypher dinámico	5 productos de Cocina

3.7 Modelo de Lenguaje Local (Item 1.8)

Se utilizó Qwen 2.5 3B Instruct de Alibaba Cloud como modelo de lenguaje local. Qwen 2.5 es una familia de modelos de lenguaje de código abierto lanzada en 2024. La variante 3B Instruct fue fine-tuneada específicamente para seguir instrucciones en formato de chat con roles system/user/assistant. La elección se justifica por:

- Cumplimiento de la restricción: se descarga de Hugging Face y se ejecuta completamente local en Colab sin claves API
- Soporte multilingüe: maneja español de forma aceptable, suficiente para las tareas de generación del TP
- Tamaño apropiado: 3B parámetros en float16 ocupan aproximadamente 6GB de los 15GB de VRAM disponibles en la GPU T4 de Colab gratuito
- Formato instruct: entrenado para seguir instrucciones, lo que permite usar prompts de sistema con restricciones y few-shot examples
- apply_chat_template: el tokenizador incluye una función para formatear el prompt en el formato específico de Qwen con tokens especiales im_start/im_end

Parámetros de generación configurados: temperature=0.1 para respuestas más deterministas, repetition_penalty=1.3 para evitar repetición de frases, y max_new_tokens variable según la tarea (10 para clasificación, 80 para chatbot, 150 para agente ReAct).

3.8 Generación y Conversación con Memoria (Item 1.9)

Se integró el pipeline RAG en un chatbot conversacional con historial de turnos. Cada turno se guarda en una lista con la pregunta del usuario y la respuesta del asistente. La función enriquecer_consulta() detecta preguntas ambiguas que hacen referencia al turno anterior (palabras como 'esa', 'la más barata', '¿y cuál?') y agrega automáticamente el contexto del turno previo a la consulta antes de clasificarla y buscar.

El prompt del sistema instruye al LLM a responder únicamente con la información del contexto recuperado, en máximo 2 oraciones y sin inventar datos. Se implementó post-procesamiento para eliminar casos donde el modelo repite el prompt en su respuesta y truncamiento si la respuesta supera las 3 oraciones. Si no se encuentra información relevante, el sistema informa al usuario que no hay datos disponibles para esa consulta.

4. Pruebas del Ejercicio 1

Se realizaron 5 pruebas del sistema RAG completo evaluando el pipeline de principio a fin: desde la clasificación de intención hasta la generación de respuesta con el LLM. Las pruebas cubren los 3 tipos de fuentes y distintos tipos de consulta.

Prueba 1: Uso de producto (Fuente Vectorial)

Consulta: '¿Cómo uso mi licuadora para hacer smoothies?'

Intención detectada: VECTORIAL. El clasificador TF-IDF identifica correctamente las palabras 'uso' y 'licuadora' como señales de consulta vectorial. La búsqueda híbrida recupera fragmentos de FAQs de licuadora con score semántico de 0.50 y score BM25 dominante por la coincidencia exacta del término 'licuadora'. La respuesta del LLM menciona las 5 velocidades y la función pulse para smoothies.

Prueba 2: Consulta de precio con filtro numérico (Fuente Tabular)

Consulta: '¿Cuáles son las licuadoras de menos de $200?'

Intención detectada: TABULAR. Las palabras 'menos de' y '$200' disparan la clasificación tabular. SQL generado: SELECT nombre, precio_usd, stock FROM productos WHERE LOWER(nombre) LIKE '%licuadora%' ORDER BY precio_usd ASC LIMIT 5. El resultado muestra las licuadoras ordenadas por precio — la más barata es la Compacto Licuadora a $259.42, confirmando que no existen licuadoras por debajo de $200 en el catálogo.

Prueba 3: Opiniones de usuarios (Fuente Vectorial con filtro)

Consulta: '¿Qué opinan los usuarios de esta cafetera?'

Intención detectada: VECTORIAL. El chatbot detecta la palabra 'opinan' y aplica filtro filtro_fuente='resena' para buscar solo en reseñas. Se recuperan reseñas reales de la Deluxe Cafetera 2024 (P0120) con puntajes de 2/5 y 4/5. La respuesta resume las opiniones variadas encontradas, mencionando tanto experiencias positivas como negativas.

Prueba 4: Recomendación por reseñas (Fuente Vectorial)

Consulta: 'Quiero una licuadora con buenas reseñas'

Intención detectada: VECTORIAL. La búsqueda híbrida recupera reseñas de Plus Licuadora Pro (P0003) con puntaje 3/5, Licuadora (P0006) con 3/5 y Ultra Licuadora (P0008) con 3/5. La respuesta del LLM menciona los modelos disponibles con sus puntajes. Esta prueba es interesante porque muestra que el sistema puede manejar consultas de recomendación aunque el dataset no tenga productos con puntajes muy altos.

Prueba 5: Relaciones en el grafo (Fuente Grafos)

Consulta: '¿Qué productos están relacionados con la categoría Cocina?'

Intención detectada: GRAFOS. La función buscar_grafo() detecta la palabra 'categoría' y 'Cocina' y ejecuta el Cypher: MATCH (p:Producto)-[:PERTENECE_A]->(c:Categoria {nombre: $categoria}) RETURN p.nombre, c.nombre, p.precio LIMIT 10. Se recuperan 5 productos: Freidora de Aire Plus ($1156.62), Turbo Deshidratador ($2095.80), Ultra Deshidratador ($1140.36), Deshidratador ($297.84) y Plus Deshidratador 2024 ($2141.41).

5. Ejercicio 2: Agente ReAct

5.1 Paradigma ReAct

ReAct (Reasoning + Acting) es un paradigma introducido por Yao et al. (2022) donde los LLMs generan trazas de razonamiento y acciones de forma intercalada, en vez de actuar directamente o solo razonar. La idea central es que el razonamiento guía la acción y la acción actualiza el razonamiento con nueva información del entorno.

En cada iteración del ciclo ReAct el agente: (1) genera un Thought que explica su razonamiento sobre qué herramienta usar y por qué, (2) especifica una Action eligiendo una de las herramientas disponibles, (3) proporciona un Action Input con la consulta para esa herramienta, y (4) recibe una Observation con el resultado de la herramienta. Este ciclo se repite hasta que el agente considera que tiene suficiente información para formular una Final Answer.

La ventaja de ReAct sobre el RAG simple es que el agente puede combinar múltiples herramientas en una sola respuesta, adaptar su estrategia según los resultados intermedios, y manejar consultas complejas que requieren información de varias fuentes. Según Wang et al. (2025), los agentes basados en LLMs con herramientas representan una evolución natural de los sistemas RAG hacia sistemas más autónomos y flexibles.

5.2 Implementación con LangChain

Se encapsuló la lógica de cada fuente de datos en herramientas independientes usando el decorador @tool de langchain_core. Este decorador convierte funciones Python estándar en herramientas que el agente puede invocar, registrando el nombre de la función y su docstring como metadatos que el agente usa para decidir cuándo usar cada herramienta.

La implementación del ciclo ReAct se realizó manualmente dado que la versión de LangChain disponible en Colab (1.3.6) eliminó initialize_agent y AgentType de las versiones anteriores. La implementación manual cumple exactamente el mismo propósito: parsear el output del LLM para extraer Action y Action Input, invocar la herramienta correspondiente, y acumular las observaciones en el contexto para la siguiente iteración.

5.3 Herramientas Implementadas

Se implementaron las 4 herramientas requeridas por la consigna, cada una encapsulando la lógica de una fuente de datos diferente:

Herramienta	Descripción
doc_search	Búsqueda híbrida semántica + BM25 con ReRank en documentos, manuales, FAQs y reseñas
table_search	Generación dinámica de SQL con LLM y fallback por palabras clave para datos tabulares
graph_search	Consultas basadas en palabras clave al grafo Neo4j de categorías, marcas y sucursales
analytics_tool	Análisis estadísticos y generación de gráficos matplotlib de ventas, pagos y devoluciones

5.4 Prompt del Sistema del Agente

Se diseñó un prompt de sistema que incluye la descripción de cada herramienta con indicaciones explícitas de cuándo usar cada una (MAYÚSCULAS para palabras clave), el formato obligatorio Thought/Action/Action Input o Final Answer, e instrucciones específicas como 'Action Input debe ser texto simple, nunca JSON' para evitar errores de parsing.

El prompt se incluye en cada iteración del ciclo junto con el contexto acumulado de las observaciones anteriores. Esto permite al agente ver qué herramientas ya usó y qué información recopiló, guiando su razonamiento hacia la respuesta final.

5.5 Mecanismos de Robustez

Se implementaron tres mecanismos para hacer el agente más robusto ante las limitaciones del modelo pequeño. Primero, corrección flexible de nombres de herramientas por substring: si 'analytic' aparece en el action se corrige a 'analytics_tool', si 'table' aparece se corrige a 'table_search', etc. Segundo, parseo de Action Input con mayúscula y minúscula ya que el modelo alterna entre 'Action Input:' y 'Action input:'. Tercero, fallback por palabras clave de la consulta original cuando el nombre de la herramienta no se reconoce.

6. Pruebas del Ejercicio 2

Se realizaron 5 pruebas del agente ReAct cubriendo las 4 herramientas disponibles. Para cada prueba se muestra el ciclo Thought-Action-Observation y la respuesta final generada.

Prueba 1: Uso de producto (doc_search)

Consulta: '¿Cómo uso mi licuadora para hacer smoothies?'

El agente razona en el Thought que necesita buscar instrucciones de uso y selecciona doc_search. Recupera fragmentos del manual técnico con información sobre las 5 velocidades, función pulse y diseño ergonómico. Llega a Final Answer en 2-4 iteraciones con instrucciones prácticas para preparar smoothies mencionando el control de velocidad para texturas sedosas.

Prueba 2: Precios y stock disponible (table_search)

Consulta: '¿Cuáles son las licuadoras disponibles y cuánto cuestan?'

El agente selecciona table_search luego de un primer intento fallido con graph_search. La herramienta genera y ejecuta SQL que devuelve: Compacto Licuadora ($259.42, 75 unidades), Licuadora ($283.63, 108 unidades), Plus Licuadora Pro ($329.07, 97 unidades) y modelos adicionales. La respuesta final lista las opciones disponibles con sus precios ordenados de menor a mayor.

Prueba 3: Opiniones de usuarios (doc_search)

Consulta: '¿Qué opinan los usuarios de la cafetera?'

El agente usa doc_search con múltiples iteraciones buscando reseñas específicas de cafeteras. Recupera reseñas de la Deluxe Cafetera 2024 con puntajes variados: Francisco Castillo (4/5, satisfecho), Renata García (2/5, problemas de durabilidad), Martina Medina (3/5, podría ser mejor). La respuesta final resume la variabilidad de opiniones mencionando que algunos usuarios están satisfechos mientras otros reportan problemas de calidad.

Prueba 4: Relaciones en categorías (graph_search)

Consulta: '¿Qué productos hay en la categoría Cocina?'

El agente selecciona graph_search y en el Action Input incluye palabras clave de cocina. La función buscar_grafo() detecta 'cocina' en la consulta y ejecuta el Cypher correspondiente. Recupera: Freidora de Aire Plus ($1156.62), Turbo Deshidratador ($2095.80), Ultra Deshidratador ($1140.36), Deshidratador ($297.84) y Plus Deshidratador 2024 ($2141.41). La respuesta Final Answer lista estos productos correctamente.

Prueba 5: Análisis estadístico con gráfico (analytics_tool)

Consulta: '¿Cuál es la distribución de métodos de pago?'

El agente intenta usar 'analytic_tool' (sin la s), que es corregido automáticamente a 'analytics_tool'. La herramienta ejecuta SELECT metodo_pago, COUNT(*) FROM ventas GROUP BY metodo_pago, genera un gráfico de barras en matplotlib guardado en /content/grafico_pagos.png y devuelve los datos: Tarjeta Débito (1.315), Tarjeta Crédito 6 cuotas (1.286), Mercado Pago (1.285), Transferencia Bancaria (1.265), Tarjeta Crédito 1 pago (1.236), Tarjeta Crédito 3 cuotas (1.228), Tarjeta Crédito 12 cuotas (1.196) y Efectivo (1.189). La distribución es notablemente uniforme entre todos los métodos.

7. Análisis de Limitaciones y Mejoras Posibles

Durante el desarrollo e implementación del sistema se identificaron varias limitaciones relacionadas principalmente con el uso de un modelo de lenguaje pequeño. A continuación se describen las limitaciones detectadas y las mejoras propuestas:

7.1 Limitaciones del Modelo de Lenguaje (3B parámetros)

El principal desafío del sistema es la limitación inherente del modelo Qwen 2.5 3B para seguir instrucciones estrictas. Se identificaron cuatro problemas principales durante las pruebas:

- Generación fuera del contexto: el modelo ocasionalmente agrega datos no presentes en el contexto recuperado, especialmente en el chatbot. Se mitigó con prompts restrictivos (máximo 2 oraciones, SOLO con info del contexto) pero no se elimina completamente
- SQL/Cypher con errores sintácticos: genera consultas con palabras clave pegadas sin espacios (WHERELOWER, ASCLIMIT). Se implementó limpiar_sql() con post-procesamiento pero algunos casos requieren el fallback
- Errores tipográficos en herramientas: escribe variantes incorrectas como 'analytic_tool' en vez de 'analytics_tool'. Se solucionó con detección por substring
- Generación en portugués: ocasionalmente usa propiedades en portugués (p.nome en vez de p.nombre) en Cypher. Limitación no resuelta, documentada como problema conocido

Mejora: usar un modelo de 7B+ parámetros (Qwen 2.5 7B, Llama 3.1 8B) que mejora significativamente la adherencia a instrucciones. El costo es mayor uso de VRAM (~14GB para 7B en float16), lo que requeriría una GPU más potente o cuantización a 4 bits con bitsandbytes.

7.2 Calidad de los Embeddings

El modelo paraphrase-multilingual-MiniLM-L12-v2, aunque eficiente, produce scores de similitud semántica relativamente bajos (0.3-0.5) para consultas en español sobre electrodomésticos. Esto resulta en que BM25 domina los resultados de la búsqueda híbrida en la mayoría de los casos, perdiendo parte del beneficio de la búsqueda semántica.

Mejora: usar un modelo de embedding más grande como multilingual-e5-large (560M parámetros, vectores de 1024 dimensiones) o BAAI/bge-m3 que fue diseñado específicamente para retrieval multilingüe con alta calidad. Alternativamente, se podría hacer fine-tuning del modelo de embedding con pares de preguntas y respuestas del dataset para mejorar la calidad en el dominio específico.

7.3 Sistema de ReRank

El sistema de ReRank actual usa una media ponderada simple (50%/50%) entre scores semánticos y BM25. No se implementó un cross-encoder para reranking más sofisticado, que evaluaría la relevancia de cada par (consulta, documento) de forma más precisa que la combinación de scores independientes.

Mejora: implementar un cross-encoder como BAAI/bge-reranker-v2-m3 en una tercera etapa del pipeline: primero recuperar candidatos con búsqueda híbrida, luego reranquear con el cross-encoder. Los cross-encoders son más precisos pero más lentos, por lo que se aplican solo sobre los top-k candidatos.

7.4 Memoria Conversacional

La memoria actual almacena los últimos turnos como texto plano y usa detección de palabras ambiguas para enriquecer la consulta. Esta aproximación simple puede contaminar el contexto cuando hay cambios de tema bruscos o cuando la conversación es larga. Además, el historial no se usa directamente en el prompt del LLM generador, solo para enriquecer la consulta de búsqueda.

Mejora: implementar memoria semántica usando embeddings para recuperar solo los turnos relevantes a la consulta actual, similar al enfoque de memoria episódica descrito en Wang et al. (2025). También se podría incorporar el historial directamente en el prompt del LLM para generar respuestas más coherentes con el contexto conversacional.

7.5 Agente ReAct: Convergencia

El agente actualmente usa un máximo de 5 iteraciones por consulta. Con el modelo pequeño, frecuentemente no llega a Final Answer espontáneamente y cae en el mecanismo de respuesta por contexto acumulado. Además, el agente a veces repite la misma herramienta con variantes de la misma consulta sin obtener nueva información.

Mejora: implementar detección de loops (cuando el agente repite la misma Action con input similar), aumentar la claridad del prompt del sistema con ejemplos de cuándo usar Final Answer, y considerar un modelo más grande para el agente ReAct mientras se mantiene el modelo pequeño para las tareas de generación SQL/Cypher donde es suficiente.

8. Conclusiones

Se desarrolló exitosamente un asistente virtual para ElectroHogar que implementa los dos ejercicios solicitados por la consigna. El Ejercicio 1 implementa un sistema RAG completo con tres bases de datos heterogéneas, búsqueda híbrida BM25 + semántica con ReRank, clasificador de intención con 93% de accuracy, y generación de respuestas con memoria conversacional. El Ejercicio 2 extiende el sistema con un agente ReAct con 4 herramientas especializadas que demuestra el ciclo Thought-Action-Observation.

Todo el sistema se ejecuta completamente local en Google Colab sin dependencia de APIs externas, cumpliendo con la restricción principal de la consigna. Las tecnologías utilizadas (ChromaDB, Neo4j, SQLite, Qwen 2.5, LangChain, sentence-transformers, scikit-learn) son todas de código abierto y reproducibles.

Las limitaciones principales están asociadas al uso de un modelo de lenguaje pequeño de 3B parámetros. Si bien este modelo cumple con el requisito de ejecución local en hardware gratuito, presenta restricciones en la adherencia a instrucciones, la calidad del SQL/Cypher generado y la tendencia a generar información fuera del contexto. Estas limitaciones son documentadas con soluciones concretas y representan áreas claras de mejora futura.

El proyecto demuestra que es posible implementar un asistente virtual RAG completo con múltiples fuentes de datos heterogéneas usando exclusivamente tecnologías de código abierto, ejecutando en hardware gratuito de Google Colab. La arquitectura modular con clasificador de intención + múltiples bases de datos es extensible a otros dominios y datasets más grandes.

9. Pruebas Interactivas

Al final del notebook se incluyeron dos celdas opcionales comentadas que permiten realizar pruebas interactivas adicionales sin necesidad de modificar el código principal. Estas celdas están disponibles para que el corrector pueda probar el sistema con consultas propias.

Prueba Interactiva del Chatbot

Se descomentó la celda opcional del chatbot y se realizó la siguiente prueba:

Consulta: "¿Qué lavarropas tienen disponibles?"

Intención detectada: TABULAR. El sistema generó SQL correcto y encontró lavarropas disponibles: Deluxe Lavarropas Automático 2024 ($191.93), Lavarropas Automático ($708.55) y otros modelos. La respuesta del LLM resumió las opciones disponibles.

Prueba Interactiva del Agente ReAct

Se descomentó la celda opcional del agente y se realizó la siguiente prueba:

Consulta: "¿Qué productos hay en Climatización?"

El agente seleccionó graph_search con input "categoría climatización" y en solo 2 iteraciones obtuvo la respuesta correcta: Compacto Ventilador de Torre ($1217.47), Deluxe Ionizador ($1390.51), Max Ionizador ($1585.04) y otros productos de la categoría. Llegó a Final Answer sin necesitar iteraciones adicionales.

10. Bibliografía

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., ... & Wen, J. R. (2025). A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science. arXiv:2308.11432v7.

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., ... & Gui, T. (2023). The Rise and Potential of Large Language Model Based Agents: A Survey. arXiv:2309.07864v3.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903.

Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333-389.

Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., & Wu, X. (2023). Unifying Large Language Models and Knowledge Graphs: A Roadmap. IEEE Transactions on Knowledge and Data Engineering. arXiv:2306.08302.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019. arXiv:1908.10084.

Materiales de la cátedra TUIA NLP 2026: Unidad 4 (LLMs, Prompting y RAG), Unidad 5 (Bases de Datos Vectoriales, BM25 y Grafos), Unidad 6 (Chatbots y Sistemas Conversacionales) y Unidad 7 (Agentes LLM y ReAct).

Chroma documentation: https://docs.trychroma.com

Neo4j Cypher Manual: https://neo4j.com/docs/cypher-manual/

LangChain documentation: https://python.langchain.com/docs

Qwen 2.5 Technical Report: https://huggingface.co/Qwen/Qwen2.5-3B-Instruct

sentence-transformers documentation: https://www.sbert.net

scikit-learn documentation: https://scikit-learn.org/stable/